Machine Learning Lab 14

Name: Dhruv Maheshwari – PES2UG23CS173 Sem 5 – C Date: 18th Nov 2025

Introduction

The objective of this lab was to design, train, and evaluate a Convolutional Neural Network (CNN) using PyTorch to classify images of hand gestures into three classes: rock, paper, and scissors. We used the publicly available Rock-Paper-Scissors dataset and built a small CNN from scratch, trained it on the dataset, and measured test accuracy.

Model Architecture

- Convolutional Backbone: Three Conv blocks with kernel size 3 and padding 1; channels: 3→16→32→64. Each conv is followed by ReLU and MaxPool2d(2), progressively reducing spatial dimensions from 128→64→32→16.
- Classifier Head: Flatten → Linear(64×16×16=16384 → 256) → ReLU → Dropout(0.3) → Linear(256 → 3).
- Total downsampling: three max-pooling operations; feature map size at classifier input is 64×16×16.

Training and Performance

- Loss Function: CrossEntropyLoss
- Optimizer: Adam
- Learning Rate: 0.001
- Epochs: 10
- Batch Size: 32
- Data Preprocessing: Resize to 128×128, Normalize with mean=[0.5,0.5,0.5], std=[0.5,0.5,0.5]
- Split: 80% train / 20% test (random_split with fixed seed)
- Final Test Accuracy (this run): 98.17%

Cell 1 Output

```
Path to dataset files: /kaggle/input/rockpaperscissors
```

Cell 2 Output

```
Copied: rock
Copied: paper
Copied: scissors
```

Cell 3 Output

```
Using device: cpu
```

Cell 4 Output

```
Classes: ['paper', 'rock', 'scissors']
Total images: 2188
Training images: 1750
Test images: 438
```

Cell 5 Output

```
RPS_CNN(
  (conv_block): Sequential(
    (0): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (fc): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=16384, out_features=256, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.3, inplace=False)
    (4): Linear(in_features=256, out_features=3, bias=True)
  )
)
```

Cell 6 Output

```
Epoch 1/10, Loss = 0.6608
Epoch 2/10, Loss = 0.2009
Epoch 3/10, Loss = 0.1032
Epoch 4/10, Loss = 0.0557
Epoch 5/10, Loss = 0.0260
Epoch 6/10, Loss = 0.0132
Epoch 7/10, Loss = 0.0114
Epoch 8/10, Loss = 0.0099
Epoch 9/10, Loss = 0.0131
Epoch 10/10, Loss = 0.0060
Training complete!
```

Cell 7 Output

```
Test Accuracy: 97.95%
```

Cell 8 Output

```
Model prediction for /content/dataset/paper/VfIYGbOFu36N6MDw.png: paper
```

Cell 9 Output

```
Randomly selected images:
Image 1: /content/dataset/paper/wytuIz48uLeDoGqz.png
Image 2: /content/dataset/scissors/xPDNBvgWZrdijzTm.png

Player 1 shows: paper
Player 2 shows: scissors

RESULT: Player 2 wins! scissors beats paper
```

Conclusion and Analysis

The model achieved strong performance (97.95% test accuracy), indicating this compact CNN is sufficient for the dataset. Challenges included ensuring the dataset path worked across environments and filtering out an extra folder (rps-cv-images) present in the Kaggle package. Potential improvements:

- Add data augmentation (RandomHorizontalFlip, ColorJitter) to improve generalization and robustness.
- Introduce learning rate scheduling or early stopping to further stabilize training.
- Try a slightly deeper network or pretrained features (e.g., ResNet18) with fine-tuning.

Summary Results

- Architecture: 3 Conv blocks (3->16->32->64), kernel 3, padding 1, maxpool 2 after each.
- Classifier: Flatten -> Linear(16384->256) -> ReLU -> Dropout(0.3) -> Linear(256->3)
- Hyperparameters: Adam lr=0.001, CrossEntropyLoss, batch_size=32, epochs=10
- Final Test Accuracy (example run): ~98%
- Improvement ideas: data augmentation (RandomHorizontalFlip), early stopping, learning rate scheduling.